Linux Booting Process & Run Levels

Comprehensive Study Notes | PGCP-ITISS

1. Introduction to the Linux Boot Process

When a Linux system is powered on, it goes through a well-defined sequence of events before the user can log in and use the system. Understanding this sequence is fundamental for system administration, troubleshooting, and performance tuning. The entire process is hardware-agnostic at the kernel level and distribution-independent in its design principles.

Key Concept	*The boot process transforms a powered-off machine into a fully running operating system through a series of layered stages — each stage hands control to the next.*

2. Stages of the Linux Boot Process

The Linux boot process consists of six major stages:

Stage 1 — BIOS / UEFI (Power-On Self Test)

This is the very first stage. It is handled entirely by firmware stored on the motherboard chip — not by Linux.

- BIOS (Basic Input/Output System) is the traditional firmware; UEFI (Unified Extensible Firmware Interface) is its modern replacement.
- When the system is powered on, the CPU executes the firmware code stored in non-volatile memory (ROM/Flash).
- The firmware performs the POST (Power-On Self Test) — checking that essential hardware components (RAM, CPU, keyboard, storage) are functioning correctly.
- After POST, the firmware locates the boot device (hard disk, SSD, USB, network) based on the configured boot order.
- It reads the MBR (Master Boot Record) or the GPT/EFI partition (in UEFI systems) from the boot device.

BIOS vs UEFI	*BIOS uses MBR (first 512 bytes of disk). UEFI uses GPT and reads directly from an EFI System Partition (ESP). UEFI supports secure boot, larger disks, and faster initialization.*

Stage 2 — Boot Loader

The boot loader is a small program loaded by the firmware. Its job is to load the Linux kernel into memory.

- Common boot loaders: GRUB2 (GRand Unified Bootloader version 2) is the most widely used across distributions.
- Other boot loaders include SYSLINUX, LILO (legacy), systemd-boot, and rEFInd.
- GRUB2 is loaded in two stages:
  - Stage 1: Tiny code that fits in MBR (446 bytes); loads Stage 1.5 or Stage 2.
  - Stage 2: Full GRUB2 environment, usually stored in /boot/grub/ — reads configuration, presents the boot menu.
- The boot loader reads its configuration file (e.g., /boot/grub/grub.cfg) and presents a menu if multiple kernels or operating systems are available.
- After the user selects an entry (or the default timeout), the boot loader loads the kernel image and the initrd/initramfs image into RAM, then transfers control to the kernel.

GRUB2 File / Directory	Purpose
/boot/grub/grub.cfg	Auto-generated main configuration file (do not edit manually)
/etc/default/grub	User-editable settings (timeout, default entry, kernel params)
/etc/grub.d/	Scripts that generate grub.cfg when update-grub or grub-mkconfig runs
/boot/vmlinuz-*	Compressed kernel image
/boot/initrd.img-* or initramfs-*.img	Initial RAM disk / RAM filesystem image

Stage 3 — Kernel Initialization

The Linux kernel is the heart of the operating system. Once loaded into RAM, it takes control from the boot loader.

- The kernel is loaded as a compressed image (vmlinuz). It decompresses itself into memory first.
- The kernel initializes the CPU, memory management (MMU), and hardware abstraction layer.
- It mounts the initramfs (initial RAM filesystem) as a temporary root filesystem. This is needed because real drivers for the actual disk may not yet be loaded.
- The initramfs contains essential modules and scripts to detect and mount the real root filesystem.
- The kernel detects and initializes hardware devices — loading appropriate kernel modules (drivers) from the initramfs or from disk.
- Once the real root filesystem is mounted, the kernel discards initramfs and switches to the real root.
- Finally, the kernel starts the very first user-space process: the init system (PID 1).

What is initramfs?	*initramfs is a compressed cpio archive embedded in RAM. It provides a minimal root filesystem early in boot so the kernel can load storage drivers and mount the actual root partition. Without it, the kernel could not access the disk containing the real OS files.*

Stage 4 — Init System (PID 1)

The init system is the first user-space process (always assigned PID 1). It is the ancestor of all other processes and is responsible for bringing the system to a usable state.

- Traditional init: SysVinit — sequential, script-based, uses /etc/inittab and /etc/rc.d/ scripts.
- Modern init: systemd — parallel, dependency-based, event-driven; the dominant init system today.
- Other init systems: Upstart (event-driven, used historically by some distributions), OpenRC (used by Gentoo and Alpine), runit, s6.
- The init system reads its configuration and starts all required services, daemons, and system processes.
- It manages the concept of run levels (SysVinit) or targets (systemd) — defining what services run at what state.

Stage 5 — Services / Daemons Start

- Networking services (bringing up network interfaces, obtaining IP addresses)
- Logging services (syslog, journald)
- Cron/scheduler services
- Display manager / login manager (for graphical desktops)
- Application services (web servers, databases, etc., if configured)
- Each service is started based on dependencies — a service waits for its dependencies to be ready first (systemd enforces this automatically).

Stage 6 — Login Prompt

- Text-based login: getty process opens a terminal (TTY) and presents a login prompt.
- Graphical login: a display manager (e.g., GDM, SDDM, LightDM) presents a graphical login screen.
- After successful authentication, the user's shell or desktop environment is launched.

3. Boot Process — Quick Reference Summary

Stage	Name	Component	What Happens
1	Firmware	BIOS / UEFI	POST; hardware check; locate boot device; load MBR/EFI

Stage	Name	Component	What Happens
2	Boot Loader	GRUB2 / others	Present boot menu; load kernel + initramfs into RAM
3	Kernel Init	Linux Kernel	Decompress; initialize hardware; mount initramfs; start PID 1
4	Init System	systemd / SysVinit	Read configuration; start services per run level / target
5	Services	Daemons & Processes	Network, logging, display manager, application services
6	Login	getty / Display Mgr	Present login prompt (text or graphical); user session starts

4. Run Levels in Linux

A run level defines the operational state of the system — which services and processes are active. Run levels are a concept from traditional SysVinit; systemd maps them to named targets for backward compatibility.

4.1 Standard Run Levels (SysVinit)

There are 7 run levels, numbered 0 through 6. Their exact meaning for levels 2–5 can vary slightly between distributions, but the extremes (0, 1, 6) are universally consistent.

Level	Name	Description
0	Halt / Power Off	Shuts down all services and powers off the machine. Never set as default.
1	Single-User Mode	Minimal mode with only the root user. No networking. Used for maintenance, password recovery, and system repairs.
2	Multi-User (No NFS)	Multi-user mode without network file sharing. Networking may or may not be active depending on the distribution.
3	Multi-User + Network	Full multi-user mode with networking enabled. Text-mode (command-line) only — no graphical interface. Common default on servers.
4	Undefined / Custom	Not defined by the standard; reserved for custom use by administrators or specific distributions.
5	Multi-User + GUI	Full multi-user mode with networking and a graphical desktop environment. The default on most desktop distributions.
6	Reboot	Shuts down all services and reboots the machine. Never set as default.

Important Rule	*Run levels 0 and 6 must never be set as the system default. Setting either as default would result in an infinite shutdown/reboot loop.*

4.2 Key Files and Directories (SysVinit)

- /etc/inittab — The main configuration file for SysVinit. It defines the default run level and the actions to take at each run level.
- /etc/rc.d/ or /etc/init.d/ — Contains shell scripts that start and stop individual services.
- /etc/rc0.d/ through /etc/rc6.d/ — Directories, one per run level. Contain symbolic links to service scripts in /etc/init.d/. Links prefixed with S (Start) are executed to start services; links prefixed with K (Kill) are executed to stop services. The number after S or K determines execution order.
- /etc/rcS.d/ — Scripts run during system initialization regardless of run level.

Naming Convention	*S20networking means: Start this service, run it 20th in order. K80networking means: Kill (stop) this service, run it 80th. Lower numbers run first.*

5. systemd Targets (Modern Equivalent of Run Levels)

systemd replaced run levels with named units called targets. Each target represents a system state, with a set of dependencies that must be satisfied. Targets are more flexible and expressive than numeric run levels.

5.1 Run Level to systemd Target Mapping

Run Level	systemd Target	Purpose
0	poweroff.target	Shut down and power off the system
1	rescue.target	Single-user / rescue mode with minimal services
2	multi-user.target	Multi-user, no graphical display (some distros map 2, 3, 4 here)
3	multi-user.target	Multi-user, networking, text-mode — most common server default
4	multi-user.target	Same as 3 by default; customizable
5	graphical.target	Multi-user, networking, and graphical login screen
6	reboot.target	Reboot the system

5.2 Other Important systemd Targets

Target	Description
default.target	Symbolic link to the default target (usually graphical.target or multi-user.target). Determines what state the system boots into.
emergency.target	Bare-bones shell with read-only root. More minimal than rescue.target. Used for critical repairs.
sysinit.target	Early system initialization — mounts filesystems, activates swap, sets up kernel parameters.
basic.target	After sysinit.target — sets up timers, sockets, paths, and basic units before other services.
network.target	Indicates that network configuration has been applied (not necessarily that the network is fully up).
network-online.target	Indicates that the network is fully up and connected. Services that need internet use this.

Target	Description
shutdown.target	Pulled in during shutdown to coordinate stopping of services.
halt.target	Halt the system without powering off.

5.3 Managing Targets with systemctl

systemctl is the primary command for interacting with systemd.

Command	What it Does
systemctl get-default	Show the current default target (boot state)
systemctl set-default multi-user.target	Set the system to boot to text-mode (like run level 3)
systemctl set-default graphical.target	Set the system to boot to graphical mode (like run level 5)
systemctl isolate rescue.target	Immediately switch to rescue mode (like changing to run level 1)
systemctl isolate multi-user.target	Switch to multi-user text mode immediately
systemctl list-units --type=target	List all currently loaded targets
systemctl poweroff	Shut down and power off (run level 0)
systemctl reboot	Reboot the system (run level 6)
systemctl rescue	Enter rescue mode (run level 1)
systemctl emergency	Enter emergency mode (more minimal than rescue)

6. Changing Run Levels / Targets

6.1 Temporarily Changing the Run Level / Target

Changes take effect immediately but do not persist after reboot.

- SysVinit command: init <run_level> Example: init 3 switches to run level 3 immediately.
- SysVinit command: telinit <run_level> telinit is a wrapper around init for safe level switching.
- systemd command: systemctl isolate <target> Example: systemctl isolate multi-user.target

6.2 Setting the Permanent Default Run Level / Target

The change will take effect on the next reboot and all subsequent boots.

- SysVinit: Edit /etc/inittab — find the line beginning with id: and change the number. Example: id:5:initdefault: (sets default to run level 5)

- systemd: Use systemctl set-default <target> Example: systemctl set-default graphical.target

6.3 Booting into a Specific Run Level at Boot Time

You can override the default run level for a single boot session from the GRUB menu:

- At the GRUB boot menu, highlight the desired entry and press 'e' to edit.
- Find the line starting with linux or linuxefi (the kernel line).
- At the end of that line, append the desired run level number (e.g., 1 for single-user, 3 for text mode) or the full systemd target name (e.g., systemd.unit=rescue.target).
- Press Ctrl+X or F10 to boot with that change for this session only.

7. Single-User Mode (Run Level 1 / rescue.target)

Single-user mode is a special minimal operational state primarily used for system maintenance and recovery. It is the equivalent of 'safe mode' in Linux.

When is Single-User Mode Used?

- Resetting a forgotten root password
- Repairing a corrupted filesystem
- Fixing misconfigured files (e.g., /etc/fstab) that prevent normal boot
- Running low-level disk diagnostics and repairs
- Troubleshooting services that prevent the system from reaching multi-user mode

Characteristics of Single-User Mode

- Only the root user can log in — no other users are active.
- Networking is disabled.
- Most non-essential services and daemons are stopped.
- The root filesystem may be mounted read-only initially; remount as read-write if needed: mount -o remount,rw /
- Only the most essential system processes are running.

8. Shutdown and Reboot Commands

Command	Effect
shutdown -h now	Immediately halt and power off (all users notified if any)
shutdown -h +10	Power off after 10 minutes, with warning to logged-in users
shutdown -r now	Immediately reboot the system

Command	Effect
shutdown -c	Cancel a pending scheduled shutdown
poweroff	Shut down and power off (equivalent to run level 0)
reboot	Reboot the system (equivalent to run level 6)
halt	Halt the CPU without powering off
init 0	SysVinit: switch to run level 0 (power off)
init 6	SysVinit: switch to run level 6 (reboot)
systemctl poweroff	systemd: power off the system
systemctl reboot	systemd: reboot the system

9. Boot Messages and Logging

During and after boot, the system generates messages that help diagnose problems.

- dmesg — Displays kernel ring buffer messages, including hardware detection messages generated during boot. Extremely useful for diagnosing hardware issues.
- journalctl -b — (systemd systems) Shows all journal messages from the current boot session.
- journalctl -b -1 — Shows journal messages from the previous boot session — useful when diagnosing a crash.
- /var/log/boot.log — Traditional log file containing boot messages (may vary by distribution).
- /var/log/syslog or /var/log/messages — General system log; contains messages from many services after boot.
- systemctl status — Shows a summary of the system state, active services, and recent journal entries.

10. Key Concepts Summary

Term	Definition
POST	Power-On Self Test — firmware-level hardware verification before boot begins.
MBR	Master Boot Record — first 512 bytes of a disk; contains the boot loader stage 1 and partition table. Used with BIOS.
GPT	GUID Partition Table — modern partition scheme used with UEFI. Supports larger disks and more partitions than MBR.
GRUB2	GRand Unified Bootloader v2 — the standard boot loader for most Linux distributions. Loads the kernel and initramfs.
vmlinuz	The compressed Linux kernel image file stored in /boot/. The 'z' indicates it is compressed with zlib/gzip.

Term	Definition
initramfs	Initial RAM Filesystem — temporary root filesystem loaded by the kernel to access drivers needed to mount the real root filesystem.
PID 1	Process ID 1 — the first user-space process started by the kernel. Always the init system (systemd or SysVinit).
systemd	Modern init system and service manager. Uses targets instead of run levels; starts services in parallel based on dependencies.
Run Level	A numeric value (0–6) defining the operational state of a SysVinit system — which services and modes are active.
Target	Named systemd unit representing a system state; the modern equivalent of a run level.
getty	A program that opens a TTY (terminal) and presents the text-mode login prompt to the user.
Display Manager	Graphical login manager (e.g., GDM, SDDM, LightDM) that provides the GUI login screen.

11. Important Points for Exams & Interviews

- Run level 0 = Halt/Shutdown; Run level 6 = Reboot. These two must never be the default run level.
- Run level 1 = Single-user/maintenance mode. No networking, only root access.
- Run level 3 = Multi-user with networking, text/CLI only (typical for servers).
- Run level 5 = Multi-user with networking and GUI (typical for desktops).
- systemd uses targets (e.g., graphical.target, multi-user.target) instead of numeric run levels.
- PID 1 is always the init process — either systemd or SysVinit.
- The kernel starts with initramfs to load necessary drivers before mounting the real root filesystem.
- GRUB2 is the boot loader; it reads /boot/grub/grub.cfg to present the boot menu.
- Use systemctl set-default to permanently change the boot target in systemd systems.
- Use systemctl isolate to immediately switch to a different target without rebooting.
- dmesg shows kernel boot messages; journalctl -b shows all systemd journal messages from current boot.
- The /etc/inittab file defines the default run level in traditional SysVinit systems.

These notes are general and apply to all Linux distributions.
Specific file paths and tool names may vary slightly across distributions, but the underlying concepts and sequence are universal.